

Education Data Hub

Dossier de Projet — Synthèse Complète

Certification Data Engineer · Février 2026

Résumé Exécutif

Ce document regroupe et condense les quatre parties du projet Education Data Hub, un hub centralisé de données éducatives françaises développé en 6 semaines suivant la méthodologie Agile Scrum. Le projet centralise des données publiques (IPS lycées, résultats baccalauréat, effectifs, financements) dans une infrastructure Azure moderne et les expose via une API REST sécurisée.

■ Durée	6 semaines · 3 sprints de 2 semaines
■ Livrables	46/51 Story Points · 90% du scope MVP livré
■ Budget	56€ (infrastructure Azure) sur 100€ alloués
■ Qualité	85% couverture de tests · Pylint 8.5/10
■ API	Déployée Azure Container Instance · 99,8% uptime
■ ■ Données	5 sources intégrées (data.gouv.fr, INSEE, OpenDataSoft)

Sommaire

Partie 1	Contexte et Avant-Projet	p. 2
Partie 2	Feuille de Route Agile	p. 4
Partie 3	Planification et Estimation	p. 6
Partie 4	Organisation des Rituels Agile	p. 8

Contexte et Avant-Projet

Analyse stratégique, périmètre fonctionnel, architecture et risques

1.1 Contexte Stratégique

Le secteur éducatif français génère d'importantes quantités de données issues de sources institutionnelles multiples (Ministère de l'Éducation Nationale, INSEE, académies). Ces données sont actuellement **dispersées, hétérogènes et difficiles d'accès**, limitant leur exploitation pour l'analyse comparative, la prise de décision des acteurs éducatifs et la recherche académique.

Expression du besoin : Centraliser les données éducatives publiques (IPS lycées, résultats bac, effectifs, financements) dans une infrastructure moderne permettant leur exploitation via Python, R et Power BI. Le système doit être automatisé, évolutif et conforme au RGPD.

1.2 Objectifs SMART

Critère	Description
Spécifique	Hub centralisé avec collecte automatisée, Data Lake Azure, pipeline ETL et API REST
Mesurable	5 sources · 100% imports automatisés · 85% tests · API documentée (OpenAPI)
Acceptable	Validation PO à chaque sprint · Répond aux besoins analystes et établissements
Réaliste	Infrastructure Azure existante · Stack Python/Azure maîtrisée · Sources publiques
Temporel	S1-S2 : Infrastructure · S3-S4 : ETL · S5-S6 : API · Livraison en 6 semaines

1.3 Périmètre Fonctionnel du MVP

Bloc 1 — Collecte automatisée

- Import mensuel depuis data.gouv.fr : IPS lycées, résultats bac, effectifs écoles/terminales
- Import mensuel depuis INSEE : financement des établissements
- Import mensuel depuis OpenDataSoft : lycées Île-de-France
- Workflows GitHub Actions (cron 1er du mois) avec gestion des doublons et alertes

Bloc 2 — Data Lake Azure (architecture Medallion)

- Couche raw/ : données brutes (CSV, JSON, XLSX)
- Couche cleaned/ : données nettoyées et normalisées (UTF-8, underscores)
- Couche tables/ : données transformées prêtes pour l'analyse

Bloc 3 — Pipeline ETL

- Script Python de nettoyage automatique (pandas)
- Suppression doublons, normalisation en-têtes, conversion UTF-8
- Conversion optionnelle CSV → Parquet via Azure Data Factory

Bloc 4 — Import SQL Azure

- 4 tables : ips_lycee, bac_par_academie, ecoles_effectifs, effectifs_tg

- Scripts modulaires (un par table) · Import automatisé GitHub Actions
- Conformité RGPD : registre des traitements, suppression comptes inactifs

Bloc 5 — API REST FastAPI

- Endpoints : GET /api/ips_lycee · GET /api/ips_lycee/{code}
- Authentification JWT (tokens 30 min) · Mots de passe bcrypt
- Documentation OpenAPI/Swagger · Déployée Azure Container Instance

Bloc 6 — CI/CD & Infrastructure as Code

- GitHub Actions : tests (pytest), qualité (black, isort, pylint), build Docker, déploiement ACI
- Terraform : provisionnement automatique des ressources Azure
- Backend distant Terraform (state Azure Storage)

1.4 Architecture du Système

Source	Destination	Fréquence	Format
data.gouv.fr	Storage raw/	Mensuel	CSV
INSEE	Storage raw/	Mensuel	XLSX
OpenDataSoft	Storage raw/	Mensuel	JSON
raw/ → cleaned/	Nettoyage Python	Quotidien	CSV
cleaned/ → SQL	Import SQL	Hebdo	CSV→SQL
SQL → API	FastAPI endpoints	Temps réel	JSON

Infrastructure Azure

- Resource Group : RG-OKOTWICA-Prod
- Storage Account Data Lake Gen2 (containers : raw, cleaned, tables)
- SQL Server + Database EducationData (Basic 2 GB)
- Azure Container Instance : 1 vCPU / 1.5 GB / port 8000 HTTPS
- Sécurité : Service Principal + JWT + HTTPS TLS 1.2+ + firewall SQL

1.5 Matrice des Risques

Risque	Criticité	Stratégie d'atténuation
R1 · Changement format sources	■ Majeur	Tests parseurs · alertes · buffer format
R2 · Dépassement délai	■ Modéré	Priorisation MoSCoW · buffer 20%
R3 · Dépassement budget Azure	■ Mineur	Alertes Cost Management (<40€/mois)
R4 · Faille sécurité API	■ Majeur	JWT + sanitization SQL · OWASP Top 10
R5 · Perte compétence clé	■ Modéré	Documentation exhaustive · code review
R6 · Indisponibilité GitHub Actions	■ Modéré	Scripts exécutables en local

1.6 Conformité RGPD & Éco-responsabilité

RGPD : Les datasets publics ne contiennent aucune donnée personnelle identifiante. Les comptes utilisateurs API sont sécurisés (bcrypt, JWT 30 min, HTTPS). Registre des traitements documenté · Suppression automatique comptes inactifs >3 ans · Purge des logs >90 jours via Azure Policy.

Éco-responsabilité (RGESN) : Région Azure France Central (bas carbone), architecture serverless (Container Instance, scaling auto), formats Parquet (-60% stockage), compression JSON gzip (-50% bande passante). Microsoft Azure et GitHub certifiés énergies renouvelables.

Feuille de Route Agile

Product Backlog · 3 Sprints · MoSCoW · Indicateurs

2.1 Product Backlog — Vue d'Ensemble

Le Product Backlog contient **15 User Stories** réparties sur **3 sprints de 2 semaines**, soit **51 Story Points** au total. La priorisation suit la méthode **MoSCoW** en respectant les dépendances techniques (infrastructure → pipeline → API).

ID	User Story	Priorité	Sprint	Estimation
US1	Infrastructure Azure (Terraform)	Must Have	S1	5 SP
US2	Import data.gouv.fr (4 datasets)	Must Have	S1	3 SP
US3	Import INSEE (financement)	Must Have	S1	2 SP
US4	Import OpenDataSoft (IDF lycées)	Must Have	S1	3 SP
US5	Tests unitaires imports	Must Have	S1	3 SP
US6	Pipeline nettoyage CSV	Must Have	S2	5 SP
US7	Azure Data Factory → Parquet	Could Have	S2 (skip)	5 SP
US8	Import SQL automatisé (4 tables)	Must Have	S2	3 SP
US9	Documentation technique	Should Have	S2	2 SP
US10	Tests d'intégration	Must Have	S2	3 SP
US11	API FastAPI (endpoints)	Must Have	S3	5 SP
US12	Authentification JWT	Must Have	S3	3 SP
US13	Dockerisation de l'API	Must Have	S3	2 SP
US14	CI/CD GitHub Actions	Should Have	S3	5 SP
US15	Documentation utilisateur	Should Have	S3	2 SP

2.2 Sprint 1 — Infrastructure & Import (S1-S2)

Sprint Goal

« Mettre en place l'infrastructure Azure et automatiser l'import de toutes les sources »

- Infrastructure Azure déployée via Terraform
- 5 workflows GitHub Actions opérationnels (import mensuel)
- Données brutes stockées dans container raw/
- Tests unitaires >85% de couverture

Vélocité : 16 SP complétés / 16 SP prévus

2.3 Sprint 2 — Pipeline ETL & SQL (S3-S4)

Sprint Goal

« Nettoyer les données et les importer en base SQL pour permettre l'exploitation analytique »

- Pipeline de nettoyage CSV fonctionnel (pandas)
- 4 tables SQL opérationnelles (ips_lycee, bac, ecoles, effectifs_tg)
- Tests d'intégration validés
- Documentation technique rédigée
- **US7 (Azure Data Factory) dépriorisée — Could Have, non critique pour MVP**

Vélocité : 13 SP complétés / 18 SP prévus

2.4 Sprint 3 — API REST & Déploiement (S5-S6)

Sprint Goal

« Exposer les données via une API sécurisée et déployer automatiquement sur Azure »

- API FastAPI déployée sur Azure Container Instance
- Authentification JWT opérationnelle (bcrypt + tokens)
- Pipeline CI/CD complet (tests + Docker + déploiement ACI)
- Documentation utilisateur et OpenAPI/Swagger
- **+0,7j absorbé par buffer 20% (sous-estimation authentification JWT)**

Vélocité : 17 SP complétés / 17 SP prévus

2.5 Méthode de Priorisation MoSCoW

Catégorie	User Stories / Description
Must Have	US1-US6, US8, US10-US13 (fonctionnalités critiques MVP)
Should Have	US9 Documentation · US14 CI/CD · US15 Doc utilisateur
Could Have	US7 Azure Data Factory Parquet (nice to have)
Won't Have	Dashboard Power BI · Exports Excel · Alertes email · Streaming

2.6 Indicateurs de Suivi

Vélocité moyenne	15,3 SP/sprint (S1: 16 SP · S2: 13 SP · S3: 17 SP)
Couverture tests	S1: 85% · S2: 83% · S3: 87% (objectif >80% ■)
Bugs critiques	0 sur l'ensemble du projet
Respect délai	98% (1 jour de retard Sprint 3, absorbé)
Respect budget	56€ / 100€ alloués (-44%)
Qualité code (Pylint)	8,5/10 (objectif >8/10 ■)
Disponibilité API	99,8% uptime (objectif >99% ■)
Adoption	15 utilisateurs inscrits (objectif >10 ■)

Planification et Estimation

Équipe · Planning Poker · Calendrier · Ressources · Risques

3.1 Composition de l'Équipe

Rôle	Disponibilité	Responsabilités principales
Data Engineer (vous)	Temps plein · 7h/j · 30 j	Développement, tests, CI/CD, déploiement, documentation, Scrum Master
Product Owner (encadrant)	2h/sprint (planning + review)	Validation US, priorisation backlog, arbitrage changements
Testeur/QA (peer review)	2h/sprint	Code review via Pull Requests · tests exploratoires

RACI résumé : Le Data Engineer est Responsable et Accountable sur toutes les tâches techniques (développement, déploiement, documentation). Le Product Owner est Accountable sur la validation des US et la priorisation. Le Testeur est Responsable sur la code review.

3.2 Méthode d'Estimation : Planning Poker

Estimation basée sur la suite de Fibonacci modifiée. Chaque Story Point est converti en **0,5 jour de développement effectif** (60% dev, 20% tests, 10% doc, 10% rituels). Un buffer de **20%** est appliqué à toutes les estimations. À partir de la Rétrospective 1, un buffer supplémentaire de **30%** est recommandé pour les tâches Azure.

Valeur	Durée estimée	Exemple
1 SP	<2h	Très simple — ex : ajout d'un import simple
2 SP	2-4h	Simple — ex : script Python avec module existant
3 SP	4-8h	Moyen — ex : script + workflow GitHub Actions
5 SP	8-16h	Complexe — ex : infrastructure Terraform, API FastAPI
8 SP	>16h	Très complexe — ex : intégration multi-services inédite

3.3 Estimations par User Story

ID	User Story	SP	Jours	Complexité	Justification
US1	Infrastructure Terraform	5 SP	2,5j	Élevée	Multiples ressources Azure, state distant
US2	Import data.gouv.fr	3 SP	1,5j	Moyenne	4 fichiers, conversion UTF-8, gestion erreurs
US3	Import INSEE	2 SP	1,0j	Faible	1 fichier Excel, réutilisation module azure_upload
US4	Import OpenDataSoft	3 SP	1,5j	Moyenne	API REST, pagination 9 requêtes, agrégation JSON
US5	Tests unitaires	3 SP	1,5j	Moyenne	Fixtures, mocking, couverture >80%, CI/CD
US6	Pipeline nettoyage	5 SP	2,5j	Élevée	Pandas, normalisation, workflow GitHub Actions

US7	Data Factory (skip)	5 SP	—	Élevée	Dépriorisé (Could Have)
US8	Import SQL	3 SP	1,5j	Moyenne	4 tables, ODBC, création dynamique
US9	Documentation technique	2 SP	1,0j	Faible	README, diagrammes Mermaid, docstrings
US10	Tests intégration	3 SP	1,5j	Moyenne	Tests end-to-end, fixtures CI/CD
US11	API FastAPI	5 SP	2,5j	Élevée	Endpoints, SQL, CORS, OpenAPI
US12	Auth JWT	3 SP	1,5j	Moyenne	Table users, bcrypt, JWT, middleware
US13	Dockerisation	2 SP	1,0j	Faible	Dockerfile, ODBC Driver, .dockerignore
US14	CI/CD GitHub Actions	5 SP	2,5j	Élevée	2 workflows, OIDC Azure, déploiement ACI
US15	Documentation utilisateur	2 SP	1,0j	Faible	README API, exemples curl/Python

Total : 51 SP → 25,5 jours · Avec buffer 20% : 30,6 jours ≈ 6 semaines

3.4 Allocation des Ressources par Sprint

Sprint 1

US	Charge	Semaine	Justification
US1	5 SP / 20h	S1 Lun–Mer	Bloquant toutes les US suivantes
US2	3 SP / 12h	S1 Jeu–Ven	Dépend US1
US3	2 SP / 8h	S1 Ven	Parallèle US2
US4	3 SP / 12h	S2 Lun–Mar	Parallèle US2-US3
US5	3 SP / 12h	S2 Mer–Jeu	Dépend US2-US3-US4

Total Sprint 1 : 16 SP / 64h / 10 jours

Sprint 2

US	Charge	Semaine	Justification
US6	5 SP / 20h	S3 Lun–Mer	Pipeline critique
US7	0h (dépriorisé)	—	Could Have → skip
US8	3 SP / 12h	S3 Jeu + S4 Lun	Dépend US6
US9	2 SP / 8h	S4 Lun	Parallèle fin US8
US10	3 SP / 12h	S4 Mar–Jeu	Tests intégration

Total Sprint 2 : 13 SP / 52h / 10 jours

Sprint 3

US	Charge	Semaine	Justification
US11	5 SP / 20h	S5 Lun–Mer	API REST
US12	3 SP / 12h	S5 Jeu–Ven	Dépend US11
US13	2 SP / 8h	S6 Lun	Dockerisation
US14	5 SP / 20h	S6 Mar–Jeu	CI/CD complexe

US15	2 SP / 8h	S6 Ven	Documentation
------	-----------	--------	---------------

Total Sprint 3 : 17 SP / 68h / 10 jours

3.5 Gestion des Risques et Dépendances

Dépendances critiques : US1 bloque US2-US4 · US6 bloque US8 · US8 bloque US11 · US11 bloque US12-US13.
La chaîne critique est donc US1 → US2/3/4 → US6 → US8 → US11 → US12 → US13 → US14.

Risque	Impact	Mitigation	Plan de contingence
US1 échoue (Terraform)	Critique	Documentation officielle + tests manuels portail	+2j buffer / déploiement manuel
US7 non terminée (ADF)	Mineur	Priorisée Could Have · US8 fonctionne sans US7	Dépriorisée (réalisée)
US11 sous-estimée	Moyen	Buffer 20% intégré · tests progressifs	Simplification endpoints
US14 bloquée (OIDC)	Moyen	Documentation Microsoft suivie	Déploiement manuel temporaire

3.6 Synthèse des Ressources et ROI

Ressource	Allocation	Coût
Data Engineer	210h (30j × 7h)	Coût formation (hors scope)
Product Owner	9h (3 sprints)	Encadrant pédagogique
Testeur/QA	6h (3 code reviews)	Peer review bénévole
Infrastructure Azure	1,5 mois	56€
Total projet	225h humaines	~56€ (cloud uniquement)

Résultats de validation

■ Délai : 6 semaines + 1j (98%) ■ Budget : 56€ / 100€ alloués ■ Scope : 100% Must Have livrés ■ Qualité : 85% tests ■ Vitesse : 15,3 SP/sprint

Organisation des Rituels Agile

Scrum · Sprint Planning · Daily · Review · Rétrospective · DoD

4.1 Cadre Agile Scrum — Principes Appliqués

Le projet adopte le **framework Scrum** avec des sprints de **2 semaines** (suffisamment longs pour livrer 3-5 US, suffisamment courts pour s'adapter). Les 4 valeurs Scrum appliquées : livraison de valeur incrémentale, adaptation aux changements (backlog dynamique), transparence (rituels + Kanban), amélioration continue (rétrospectives).

Rituel	Durée / Fréquence	Participants	Objectifs
Sprint Planning	2h · Début de sprint	PO + Data Engineer + Testeur	Sélection US · Planning Poker · Sprint Goal · Décomposition tâches
Daily Stand-up	Asynchrone quotidien	Data Engineer	Commit structuré GitHub (hier / aujourd'hui / blocages)
Sprint Review	1h · Fin de sprint	PO + Data Engineer + Stakeholders	Démo live · feedback PO · mise à jour backlog
Sprint Rétrospective	1h · Fin de sprint	Data Engineer + PO (opt.)	Start/Stop/Continue · 5 Whys · actions d'amélioration

4.2 Sprint Planning (2h)

Phase 1 — QUOI ? (60 min) : Le Product Owner présente le contexte et les US prioritaires. L'équipe estime via le Planning Poker (suite Fibonacci), discute les écarts >2 SP et valide le consensus. Le Sprint Goal est formulé en une phrase.

Phase 2 — COMMENT ? (60 min) : Le Data Engineer (Scrum Master) décompose chaque US en tâches atomiques (<1 jour), identifie les dépendances et crée les Issues GitHub. La charge est vérifiée (capacité nette ≈ 60h/sprint, cible 15-18 SP).

Exemple — Sprint Planning 1 (08/01/2024)

Timing	Activité	Détails
9h00–9h10	Présentation contexte	Vision : centraliser données éducatives
9h10–9h30	Review US1–US5	Lecture et clarification critères d'acceptation
9h30–10h00	Planning Poker	US1=5, US2=3, US3=2, US4=3, US5=3 → 16 SP
10h00–10h30	Décomposition tâches	US1 : 7 tâches de 0,2 à 0,5j · Issues créées
10h30–11h00	Validation + engagement	Charge 16 SP < capacité 18 SP ■

4.3 Daily Stand-up Asynchrone

Adapté au contexte solo : la réunion quotidienne est remplacée par des **commits Git structurés** effectués en fin de journée (17h-18h). Bénéfices : traçabilité complète, transparence PO, gain de temps (7,5h économisées sur 6 semaines), discipline de formalisation quotidienne.

Format Commit Daily

[Daily] JJ/MM - <Tâche US en cours>

- Hier : ce qui a été accompli
- Aujourd'hui : ce qui sera fait
- Blocages : description ou 'Aucun'
- Avancement : X/Y tâches complétées (Z%)

Escalade des blocages : Technique → documentation officielle · Azure → support tickets · Fonctionnel → email PO (<24h réponse).

4.4 Sprint Review (1h)

Objectif : valider les US complétées avec le Product Owner via une **démonstration live**. Ordre du jour : rappel Sprint Goal (5 min) · démo US complétées (30 min) · feedback PO/stakeholders (15 min) · mise à jour Product Backlog (10 min).

Sprint	Demos réalisées
Sprint 1	Démo Terraform (portail Azure) · Démo 5 workflows d'import · Logs storage raw/ · Résultats tests 85%
Sprint 2	Démo pipeline nettoyage (avant/après) · Démo 4 tables SQL · Rapport couverture 83%
Sprint 3	Démo API live (Swagger) · Démo authentification JWT · Démo pipeline CI/CD GitHub · Uptime 99,8%

4.5 Sprint Rétrospective (1h)

Méthode **Start / Stop / Continue** complétée par **5 Whys** pour les causes racines. Vote par points (3 votes/participant) pour prioriser les actions. Les actions sont assignées, datées et suivies au sprint suivant.

Type	Action / Observation	Responsable	Statut
■ START S1	Documenter choix techniques (ADR)	Data Engineer	■ Sprint 2
■ STOP S1	Sous-estimer temps configuration Azure	Scrum Master	■ Buffer +30%
■ CONT. S1	Tests unitaires systématiques (85%)	—	Maintenu
■ START S2	Commits plus atomiques (1 commit = 1 feature)	Data Engineer	■ Immédiat
■ STOP S2	Tests manuels sans fixtures robustes	Testeur	■ Sprint 3
■ START S3	Tests de sécurité API (OWASP Top 10)	Data Engineer	À planifier V2

4.6 Definition of Done (DoD)

Une User Story n'est **Done** que si **100% des critères** suivants sont remplis. Un seul critère manquant → l'US reste **Not Done**.

■ Code

- Code fonctionnel et versionné sur GitHub (branche feature → main)
- Commits explicites (format : type(scope): message) · PEP 8 respecté

■ Tests

- Tests unitaires pytest · couverture >80% du module

- Tests intégration si applicable · CI/CD vert (pas de régression)

■ Documentation

- Docstrings Python · README mis à jour · Exemples d'utilisation fournis

■ CI/CD

- Pipeline CI/CD passe · Build Docker réussi (US11+) · Déploiement ACI (US14+)

■ Revue

- Pull Request approuvée · Feedback revieweur intégré · PRmerguée · 0 TODO bloquant

■ Acceptance

- Critères d'acceptation US validés · Démo Sprint Review · PO a validé · Issue fermée

Évolution de la DoD au fil du projet

Sprint	Évolution
Sprint 1	DoD initiale — tous les critères de base
Sprint 2	Ajout : ADR documenté pour tout choix technique majeur
Sprint 3	Ajout : Tests de sécurité API (OWASP Top 10) pour l'authentification JWT

Bilan Final du Projet

■ Points forts

- Méthodologie Agile Scrum respectée tout au long du projet (3 sprints, tous les rituels)
- MVP livré en 6 semaines : 46/51 SP (90%) · 100% des Must Have livrés
- Budget maîtrisé : 56€ sur 100€ alloués (–44%)
- Qualité constante : 85% couverture tests · 0 bug critique · Pylint 8,5/10
- CI/CD dès Sprint 1 → détection précoce des bugs, déploiements automatisés

■■ Points d'amélioration

- Buffer +30% recommandé pour toutes tâches Azure (complexité portail sous-estimée)
- Tests de charge et de sécurité à intégrer dès Sprint 3 (prévus V2)
- Architecture Decision Records (ADR) à systématiser dès le démarrage